Week 4 Submission

Peer-to-Peer Lending Matchmaking System

Regd No	25091A32G9, 25091A32K4
Problem Statement No	105
Title	Peer-to-Peer Lending Matchmaking System
Technology	Python, Flask, SQLite, NetworkX, Pandas/NumPy, ReportLab, OpenPyXL, Chart.js, Bootstrap, unittest/pytest
Data	Academic simulation using simulated test data - no real-money transactions

Where to run the code (read this first)

- **Where:** on your own laptop. Install **Python 3.10 or newer** from python.org (tick *Add Python to PATH*) and **VS Code**. Everything below is typed in the **VS Code terminal** (menu Terminal > New Terminal, or Ctrl+`).
- **Which folder:** open your **p2p_lending** folder (the one that contains **app.py**) with File > Open Folder. Every command must be run from this folder, not from a sub-folder, not in IDLE.
- **One-time setup:** run `python -m venv venv`, then activate it - Windows: `venv\Scripts\activate` (if PowerShell blocks it use Command Prompt or run `Set-ExecutionPolicy -Scope Process Bypass`), macOS/Linux: `source venv/bin/activate`. Then `pip install -r requirements.txt`.
- **Adding the files of a week:** in VS Code right-click the folder > New File, type the exact path shown in the heading (for example `routes/profiles.py`), paste the code. **[NEW]** = create the file, **[REPLACE]** = overwrite the old file completely, **[APPEND]** = paste at the very end of the existing file.
- **Web pages:** run `python app.py`, then open `http://127.0.0.1:5000` in your browser (stop the server with Ctrl+C). The pages load Bootstrap/Chart.js from the internet, so stay online.
- **Keep the weeks cumulative:** Week N needs the files of Weeks 1 to N-1. Your Week 1 zip is the starting point and none of its files is deleted later (only app.py and templates/index.html are replaced in Week 2, and config.py gets lines appended in Week 3).
- **Demo scripts** (`demo_weekN.py`) create their own throw-away databases in the data folder, so they never disturb your real database. To see the same demo data in the web pages: stop the server, delete `data/p2p_lending.db`, run `python demo_data.py`, start the server again.

Week 4: Graph-Based Matchmaking

Build a weighted bipartite graph (lenders on the left, requests/borrowers on the right, eligible pairs as edges weighted by the compatibility score), rank the edges and select the best matches without exceeding any lender's funds. **Working output: recommended lender-borrower matches.**

Assignment steps and where they are done

Assignment step	Implemented in
1. Load active lenders and loan requests	MatchingEngine.load_active_lenders(), load_pending_requests()
2. Create lender and borrower graph nodes	build_graph(): nodes L<id> (bipartite=0) and R<id> (bipartite=1) with NetworkX
3. Evaluate each possible pair	scoring.score_all() is called by run() (Week 3 engine)
4. Remove pairs that fail mandatory conditions	rejected pairs get no edge - only eligible pairs are added
5. Add weighted edges for eligible pairs	g.add_edge(lender, request, weight=compatibility_score)
6. Rank edges by compatibility score	rank_edges(): highest score first, ties by match id
7. Select the best matches	allocate(): greedy selection in rank order
8. Prevent allocations beyond a lender's available amount	capacity per lender is reduced after every allocation
9. Partial funding (optional)	MatchingEngine(conn, allow_partial=True): several lenders co-fund one loan; a request is kept only if fully covered

Files of this week

File	Action	Purpose
matching_engine.py	REPLACE	Full MatchingEngine (replaces the Week 2 skeleton)
routes/matching.py	NEW	/matching/run and /matching/recommendations
demo_week4.py	NEW	Console demo: full vs partial funding
tests/helpers.py	NEW	Helper functions used by the tests of Weeks 4-7
tests/test_week4.py	NEW	10 tests

Why every selected match can be explained

After allocation each eligible pair is stored either as **recommended** (with the allocated amount and its rank) or as **expired** with a reason: *the borrower was already funded by a higher-ranked lender*, *the lender's remaining funds cannot cover this loan*, or *the request could not be fully funded*. Requests that are fully funded change to status **matched**; every decision is written to the audit log.

How to run and check this week

1. `python tests/test_week4.py` - 10 tests, all ok

2. `python demo_week4.py` - prints the graph, ranked edges and recommended matches for full and partial funding

3. `python app.py` - open /matching/recommendations and press 'Run matching'

Expected output of demo_week4.py

```
########## FULL FUNDING ##########
Summary: {'lenders': 5, 'requests': 7, 'eligible_edges': 11, 'recommended': 5, 'requests_matched': 5, 'allocation_success_rate': 71.4,
    'total_allocated': 285000.0, 'partial_funding': False, 'seconds': 0.006}
Graph: 12 nodes, 11 weighted edges

Top-ranked edges:
  Balanced Bala       --  94.03 --> request R3 (Sunil Verma)
  Conservative Carla  --  87.68 --> request R1 (Ravi Kumar)
  Aggressive Arjun    --  86.52 --> request R2 (Meena Iyer)
  Conservative Carla  --  85.28 --> request R5 (Kiran Rao)
  Conservative Carla  --  82.93 --> request R6 (Kiran Rao)

Recommended matches:
  Request #1 Ravi Kumar   <- Conservative Carla  allocated  50,000.00  score 87.68
  Request #2 Meena Iyer   <- Aggressive Arjun    allocated  20,000.00  score 86.52
  Request #3 Sunil Verma  <- Balanced Bala       allocated  80,000.00  score 94.03
  Request #5 Kiran Rao    <- Conservative Carla  allocated  15,000.00  score 85.28
  Request #6 Kiran Rao    <- Conservative Carla  allocated 120,000.00  score 82.93

Requests left without a match:
  Request #4 Anita Das
  Request #7 Pooja Nair

Example 'not selected' explanation:
  0 vs lender band and funds) + term fit 10.71/15 (12 months) + return fit 10.0/20 (rate 9% in lender's 9-15% range). | NOT SELECTED: the
    borrower was already funded by a higher-ranked lender.

########## PARTIAL FUNDING (co-funding) ##########
Summary: {'lenders': 5, 'requests': 7, 'eligible_edges': 13, 'recommended': 7, 'requests_matched': 6, 'allocation_success_rate': 85.7,
    'total_allocated': 565000.0, 'partial_funding': True, 'seconds': 0.009}
Graph: 12 nodes, 13 weighted edges

Top-ranked edges:
  Balanced Bala       --  94.03 --> request R3 (Sunil Verma)
  Conservative Carla  --  87.68 --> request R1 (Ravi Kumar)
  Aggressive Arjun    --  86.52 --> request R2 (Meena Iyer)
  Conservative Carla  --  85.28 --> request R5 (Kiran Rao)
  Conservative Carla  --  82.93 --> request R6 (Kiran Rao)

Recommended matches:
  Request #1 Ravi Kumar   <- Conservative Carla  allocated  50,000.00  score 87.68
  Request #2 Meena Iyer   <- Aggressive Arjun    allocated  20,000.00  score 86.52
  Request #3 Sunil Verma  <- Balanced Bala       allocated  80,000.00  score 94.03
  Request #5 Kiran Rao    <- Conservative Carla  allocated  15,000.00  score 85.28
  Request #6 Kiran Rao    <- Conservative Carla  allocated 120,000.00  score 82.93
  Request #7 Pooja Nair   <- Cohort Chitra       allocated 100,000.00  score 65.5
  Request #7 Pooja Nair   <- Aggressive Arjun    allocated 180,000.00  score 64.88

Requests left without a match:
  Request #4 Anita Das

Example 'not selected' explanation:
  0 vs lender band and funds) + term fit 10.71/15 (12 months) + return fit 10.0/20 (rate 9% in lender's 9-15% range). | NOT SELECTED: the
    borrower was already funded by a higher-ranked lender.
```

[REPLACE] matching_engine.py

```
"""Week 4 - graph-based matchmaking with a weighted bipartite graph (NetworkX).

    Lender nodes (left, bipartite=0)  --edge: eligible pair, weight = compatibility score-->
    Request nodes (right, bipartite=1; each carries its borrower)

Pipeline: score pairs -> build graph from eligible pairs -> rank edges ->
greedy allocation that never exceeds a lender's available amount ->
store recommended matches with the reason for every decision.

Partial funding (optional): one loan may be co-funded by several lenders, but a
request is only kept when the lenders together cover 100% of it.
"""
import time

import networkx as nx

import audit
import profile_service
import scoring


class MatchingEngine:
    def __init__(self, conn, allow_partial=False):
        self.conn = conn
        self.allow_partial = allow_partial
        self.graph = None

    # ---- Step 1: load -------------------------------------------------------
    def load_active_lenders(self):
        return profile_service.list_lenders(self.conn, profile_status="active")

    def load_pending_requests(self):
        return profile_service.list_requests(self.conn, status="pending")

    # ---- Steps 2-5: build the bipartite graph from eligible pairs ----------
    def build_graph(self):
        g = nx.Graph()
        for lender in self.load_active_lenders():
            g.add_node(f"L{lender.lender_id}", bipartite=0, kind="lender", name=lender.full_name,
                       available=lender.available_amount)
```

```
    for req in self.load_pending_requests():
        g.add_node(f"R{req['request_id']}", bipartite=1, kind="request", name=req["borrower"],
                   amount=req["amount"], borrower_id=req["borrower_id"])
    rows = self.conn.execute(
        "SELECT match_id, lender_id, request_id, compatibility_score FROM matches"
        " WHERE status = 'recommended' AND allocated_amount = 0").fetchall()
    for row in rows:
        lender_node, request_node = f"L{row['lender_id']}", f"R{row['request_id']}"
        if lender_node in g and request_node in g:
            g.add_edge(lender_node, request_node, weight=row["compatibility_score"], match_id=row["match_id"])
    self.graph = g
    return g

# ---- Step 6: rank edges ------------------------------------------------
@staticmethod
def rank_edges(g):
    """Edges as (lender_node, request_node, data), best compatibility first."""
    edges = []
    for u, v, data in g.edges(data=True):
        lender_node, request_node = (u, v) if g.nodes[u]["bipartite"] == 0 else (v, u)
        edges.append((lender_node, request_node, data))
    edges.sort(key=lambda e: (-e[2]["weight"], e[2]["match_id"]))
    return edges

# ---- Steps 7-9: select best matches within lender capacity -------------
def allocate(self, g):
    ranked = self.rank_edges(g)
    failed = set()                       # requests that could not be fully funded
    while True:
        capacity = {n: d["available"] for n, d in g.nodes(data=True) if d["bipartite"] == 0}
        need = {n: d["amount"] for n, d in g.nodes(data=True) if d["bipartite"] == 1}
        allocations, reasons, ranks = {}, {}, {}
        for rank, (lender_node, request_node, data) in enumerate(ranked, start=1):
            match_id = data["match_id"]
            ranks[match_id] = rank
            if request_node in failed:
                reasons[match_id] = "the request could not be fully funded by the compatible lenders in this run"
            elif need[request_node] <= 0.005:
                reasons[match_id] = "the borrower was already funded by a higher-ranked lender"
            else:
                if self.allow_partial:
                    amount = min(capacity[lender_node], need[request_node])
                else:
                    amount = need[request_node] if capacity[lender_node] + 1e-9 >= need[request_node] else 0
                if amount <= 0.005:
                    reasons[match_id] = (f"the lender's remaining funds ({capacity[lender_node]:,.2f}) "
                                         f"cannot cover this loan")
                else:
                    amount = round(amount, 2)
                    allocations[match_id] = amount
                    capacity[lender_node] -= amount
                    need[request_node] -= amount
        newly_failed = set()
        if self.allow_partial:
            for request_node, remaining in need.items():
                if request_node not in failed and remaining > 0.005 and any(
                        data["match_id"] in allocations for _, r, data in ranked if r == request_node):
                    newly_failed.add(request_node)
        if not newly_failed:
            return allocations, reasons, ranks, need
        failed |= newly_failed            # free their money and try again

# ---- store results -----------------------------------------------------
def persist(self, g, allocations, reasons, ranks, need):
    conn = self.conn
    explanations = dict(conn.execute("SELECT match_id, explanation FROM matches WHERE status = 'recommended'"
                                     " AND allocated_amount = 0").fetchall())
    funded = {}
    for lender_node, request_node, data in self.rank_edges(g):
        match_id = data["match_id"]
        funded.setdefault(request_node, 0.0)
        funded[request_node] += allocations.get(match_id, 0.0)
    for match_id, rank in ranks.items():
        base = explanations[match_id]
        if match_id in allocations:
            note = f" | SELECTED (rank {rank} of {len(ranks)}): allocated {allocations[match_id]:,.2f}."
            conn.execute("UPDATE matches SET status = 'recommended', allocated_amount = ?, explanation = ?"
                         " WHERE match_id = ?", (allocations[match_id], base + note, match_id))
            audit.log(conn, "match", match_id, "RECOMMENDED", "eligible", "recommended",
                      details=f"allocated {allocations[match_id]:,.2f}")
        else:
            note = f" | NOT SELECTED: {reasons[match_id]}."
            conn.execute("UPDATE matches SET status = 'expired', explanation = ? WHERE match_id = ?",
                         (base + note, match_id))
    matched = 0
    for request_node, remaining in need.items():
        if funded.get(request_node, 0) > 0 and remaining <= 0.005:
            request_id = int(request_node[1:])
            conn.execute("UPDATE loan_requests SET status = 'matched' WHERE request_id = ?", (request_id,))
            audit.log(conn, "loan_request", request_id, "STATUS_CHANGE", "pending", "matched",
                      details=f"funded {funded[request_node]:,.2f}")
            matched += 1
    return matched

# ---- public entry point ------------------------------------------------
def run(self, rescore=True):
    started = time.perf_counter()
    if rescore:
        scoring.score_all(self.conn, allow_partial=self.allow_partial)
    g = self.build_graph()
```

```
        pending = [n for n, d in g.nodes(data=True) if d["bipartite"] == 1]
        allocations, reasons, ranks, need = self.allocate(g)
        matched = self.persist(g, allocations, reasons, ranks, need)
        audit.log(self.conn, "system", None, "MATCHING_RUN", None,
                  details=f"{len(allocations)} recommended, {matched}/{len(pending)} requests matched")
        self.conn.commit()
        return {"lenders": sum(1 for _, d in g.nodes(data=True) if d["bipartite"] == 0),
                "requests": len(pending), "eligible_edges": g.number_of_edges(),
                "recommended": len(allocations), "requests_matched": matched,
                "allocation_success_rate": round(100 * matched / len(pending), 1) if pending else 0.0,
                "total_allocated": round(sum(allocations.values()), 2),
                "partial_funding": self.allow_partial,
                "seconds": round(time.perf_counter() - started, 4)}
```

[NEW] routes/matching.py

```
"""Week 4 routes - run the matching engine and show recommended matches."""
from flask import Blueprint, flash, redirect, render_template, request, url_for

import queries
from matching_engine import MatchingEngine
from utils import db_conn

bp = Blueprint("matching", __name__, url_prefix="/matching")


@bp.route("/run", methods=["POST"])
def run():
    partial = request.args.get("partial") == "1"
    with db_conn() as conn:
        s = MatchingEngine(conn, allow_partial=partial).run()
    flash(f"Matching finished: {s['recommended']} recommended matches, {s['requests_matched']} of "
          f"{s['requests']} requests fully matched ({s['allocation_success_rate']}%), "
          f"{s['seconds']} s.", "success")
    return redirect(url_for("matching.recommendations"))


@bp.route("/recommendations")
def recommendations():
    with db_conn() as conn:
        rows = [r for r in queries.list_pairs(conn) if r["status"] in ("recommended", "accepted")
                and r["allocated_amount"] > 0]
    columns = [("request_id", "Req"), ("borrower", "Borrower"), ("lender", "Lender"), ("amount", "Loan amount"),
               ("allocated_amount", "Allocated"), ("score", "Score"), ("status", "Status"),
               ("explanation", "Why recommended")]
    return render_template(
        "table.html", title="Recommended lender-borrower matches", columns=columns, rows=rows,
        intro="Best compatibility first. A lender is never allocated more than its available amount.",
        top_actions=[{"label": "Run matching", "method": "post", "url": url_for("matching.run")},
                     {"label": "Run matching (partial funding)", "method": "post", "style": "secondary",
                      "url": url_for("matching.run", partial=1)}])
```

[NEW] demo_week4.py

```
"""Week 4 demo: weighted bipartite graph, ranking and allocation (uses data/demo.db)."""
import demo_data
import queries
from matching_engine import MatchingEngine


def run(partial):
    conn = demo_data.fresh_demo_db()
    engine = MatchingEngine(conn, allow_partial=partial)
    summary = engine.run()
    print("Summary:", summary)
    g = engine.graph
    print(f"Graph: {g.number_of_nodes()} nodes, {g.number_of_edges()} weighted edges")
    print("\nTop-ranked edges:")
    for lender_node, request_node, d in engine.rank_edges(g)[:5]:
        print(f"  {g.nodes[lender_node]['name']:<19} -- {d['weight']:>6} --> request {request_node} "
              f"({g.nodes[request_node]['name']})")
    print("\nRecommended matches:")
    for r in queries.list_pairs(conn, status="recommended"):
        if r["allocated_amount"] > 0:
            print(f"  Request #{r['request_id']} {r['borrower']:<12} <- {r['lender']:<19} "
                  f"allocated {r['allocated_amount']:>10,.2f}  score {r['score']}")
    print("\nRequests left without a match:")
    for r in conn.execute("SELECT r.request_id, u.full_name FROM loan_requests r JOIN borrowers b ON "
                          "b.borrower_id=r.borrower_id JOIN users u ON u.user_id=b.user_id "
                          "WHERE r.status='pending'"):
        print(f"  Request #{r[0]} {r[1]}")
    print("\nExample 'not selected' explanation:")
    expired = queries.list_pairs(conn, status="expired")
    if expired:
        print(" ", expired[0]["explanation"][-190:])
    conn.close()


print("########## FULL FUNDING ##########")
run(False)
print("\n########## PARTIAL FUNDING (co-funding) ##########")
run(True)
```

[NEW] *tests/helpers.py*

```
"""Shared helpers for the test files (Weeks 4-7)."""
import os
import sys
import tempfile

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import database
import profile_service as ps

_counter = [0]


def new_conn():
    path = os.path.join(tempfile.mkdtemp(), "test.db")
    database.init_db(path)
    return database.get_connection(path), path


def add_lender(conn, appetite="medium", available=100000, min_loan=5000, max_loan=100000,
               min_m=6, max_m=36, min_ret=8, max_ret=22, activate=True):
    _counter[0] += 1
    return ps.register_lender(conn, {
        "full_name": f"Lender {_counter[0]}", "email": f"lender{_counter[0]}@t.com", "password": "password123",
        "risk_appetite": appetite, "available_amount": available, "min_loan_amount": min_loan,
        "max_loan_amount": max_loan, "min_duration_months": min_m, "max_duration_months": max_m,
        "min_expected_return": min_ret, "max_expected_return": max_ret}, auto_activate=activate)


def add_borrower(conn, income=60000, obligations=5000, credit=720, years=5, activate=True):
    _counter[0] += 1
    return ps.register_borrower(conn, {
        "full_name": f"Borrower {_counter[0]}", "email": f"borrower{_counter[0]}@t.com", "password": "password123",
        "monthly_income": income, "existing_obligations": obligations, "credit_score": credit,
        "credit_history_years": years}, auto_activate=activate)
def add_request(conn, borrower_id, amount=30000, term=12, purpose="test"):
    return ps.create_loan_request(conn, borrower_id, {"amount": amount, "term_months": term, "purpose": purpose})
```

[NEW] *tests/test_week4.py*

```
"""Week 4 tests: bipartite graph, ranking, allocation limits, partial funding."""
import os
import sys
import unittest

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from helpers import add_borrower, add_lender, add_request, new_conn
import queries
from matching_engine import MatchingEngine


def recommended(conn):
    return [r for r in queries.list_pairs(conn, status="recommended") if r["allocated_amount"] > 0]


class GraphTests(unittest.TestCase):
    def test_graph_is_bipartite_with_weighted_edges(self):
        import networkx as nx
        conn, _ = new_conn()
        l1, l2 = add_lender(conn), add_lender(conn)
        b = add_borrower(conn)
        add_request(conn, b)
        engine = MatchingEngine(conn)
        engine.run()
        g = engine.graph
        self.assertTrue(nx.is_bipartite(g))
        self.assertEqual(g.number_of_nodes(), 3)
        self.assertEqual(g.number_of_edges(), 2)
        self.assertTrue(all(d["weight"] > 0 for _, _, d in g.edges(data=True)))
        ranked = MatchingEngine.rank_edges(g)
        self.assertGreaterEqual(ranked[0][2]["weight"], ranked[1][2]["weight"])


class AllocationTests(unittest.TestCase):
    def test_multiple_lenders_compete_for_one_borrower_best_wins(self):
        conn, _ = new_conn()
        exact = add_lender(conn, appetite="medium", min_ret=10, max_ret=15)
        loose = add_lender(conn, appetite="high", min_ret=10, max_ret=15)
        b = add_borrower(conn, income=45000, obligations=12000, credit=650, years=3)
        rid = add_request(conn, b, amount=60000, term=24)
        s = MatchingEngine(conn).run()
        rows = recommended(conn)
        self.assertEqual(len(rows), 1)                              # only ONE lender funds the borrower
        self.assertEqual(rows[0]["lender_id"], exact)               # the better-fitting lender wins
        self.assertEqual(s["requests_matched"], 1)
        expired = queries.list_pairs(conn, status="expired")
        self.assertEqual(expired[0]["lender_id"], loose)
        self.assertIn("higher-ranked lender", expired[0]["explanation"])

    def test_one_lender_can_fund_multiple_loans(self):
        conn, _ = new_conn()
        lender = add_lender(conn, available=100000)
        for _ in range(3):
            add_request(conn, add_borrower(conn), amount=30000)
        s = MatchingEngine(conn).run()
        self.assertEqual(s["requests_matched"], 3)
        self.assertEqual(sum(r["allocated_amount"] for r in recommended(conn)), 90000)
    def test_lender_is_never_over_allocated(self):
```

```
        conn, _ = new_conn()
        lender = add_lender(conn, available=100000)
        for _ in range(4):
            add_request(conn, add_borrower(conn), amount=30000)
        s = MatchingEngine(conn).run()
        self.assertEqual(s["requests_matched"], 3)                  # 4 x 30000 > 100000
        self.assertLessEqual(sum(r["allocated_amount"] for r in recommended(conn)), 100000)
        reasons = [r["explanation"] for r in queries.list_pairs(conn, status="expired")]
        self.assertTrue(any("remaining funds" in x for x in reasons))

    def test_no_compatible_lender(self):
        conn, _ = new_conn()
        add_lender(conn, appetite="low", min_ret=1, max_ret=5)
        add_request(conn, add_borrower(conn), amount=30000)
        s = MatchingEngine(conn).run()
        self.assertEqual(s["requests_matched"], 0)
        self.assertEqual(s["allocation_success_rate"], 0.0)
        self.assertEqual(conn.execute("SELECT status FROM loan_requests").fetchone()[0], "pending")

    def test_amount_exceeding_available_funds(self):
        conn, _ = new_conn()
        add_lender(conn, available=20000)
        add_request(conn, add_borrower(conn), amount=30000)
        self.assertEqual(MatchingEngine(conn).run()["requests_matched"], 0)

    def test_inactive_lender_is_ignored(self):
        conn, _ = new_conn()
        add_lender(conn, activate=False)
        add_request(conn, add_borrower(conn), amount=30000)
        s = MatchingEngine(conn).run()
        self.assertEqual((s["lenders"], s["requests_matched"]), (0, 0))

    def test_rerun_is_repeatable(self):
        conn, _ = new_conn()
        add_lender(conn)
        add_request(conn, add_borrower(conn))
        first = MatchingEngine(conn).run()
        second = MatchingEngine(conn).run()
        self.assertEqual(first["recommended"], second["recommended"])
        self.assertEqual(conn.execute("SELECT COUNT(*) FROM matches").fetchone()[0], 1)


class PartialFundingTests(unittest.TestCase):
    def test_two_lenders_cofund_one_request(self):
        conn, _ = new_conn()
        a, b = add_lender(conn, available=60000), add_lender(conn, available=60000)
        add_request(conn, add_borrower(conn), amount=100000)
        self.assertEqual(MatchingEngine(conn).run()["requests_matched"], 0)          # full mode fails
        s = MatchingEngine(conn, allow_partial=True).run()
        self.assertEqual(s["requests_matched"], 1)
        rows = recommended(conn)
        self.assertEqual(len(rows), 2)
        self.assertEqual(sum(r["allocated_amount"] for r in rows), 100000)
        self.assertTrue(all(r["allocated_amount"] <= 60000 for r in rows))

    def test_unfundable_request_is_rolled_back_and_frees_money(self):
        conn, _ = new_conn()
        add_lender(conn, available=60000)
        big = add_request(conn, add_borrower(conn), amount=100000)       # cannot be covered (only 60000 exists)
        small = add_request(conn, add_borrower(conn), amount=50000)
        s = MatchingEngine(conn, allow_partial=True).run()
        statuses = dict(conn.execute("SELECT request_id, status FROM loan_requests").fetchall())
        self.assertEqual(statuses[big], "pending")
        self.assertEqual(statuses[small], "matched")                      # freed money funds the smaller one
        self.assertEqual(s["total_allocated"], 50000)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Screenshots to attach to your submission

- /matching/recommendations after pressing 'Run matching' (green message with the summary).
- The same page after 'Run matching (partial funding)'.
- Terminal output of demo_week4.py (graph, ranked edges) and the 10 passing tests.